

其他算法

矩阵加速

```
//1939
// #include <iostream>
// #include <cstdio>
// #include <cstring>

// using namespace std;

// typedef long long LL;
// const int Mod = 1e9+7;
// int T, n;
// struct mat{
//     LL m[5][5];
// }Ans, base;
// inline void init() {
//     memset(Ans.m, 0, sizeof(Ans.m));
//     for(int i=1; i<=3; i++) Ans.m[i][i] = 1;
//     memset(base.m, 0, sizeof(base.m));
//     base.m[1][1] = base.m[1][3] = base.m[2][1] = base.m[3][2] = 1;
// }
// inline mat mul(mat a, mat b) {
//     mat res;
//     memset(res.m, 0, sizeof(res.m));
//     for(int i=1; i<=3; i++) {
//         for(int j=1; j<=3; j++) {
//             for(int k=1; k<=3; k++) {
//                 res.m[i][j] += (a.m[i][k] % Mod) * (b.m[k][j] % Mod);
//                 res.m[i][j] %= Mod;
//             }
//         }
//     }
//     return res;
// }
// inline void Qmat_pow(int p) {
//     while (p) {
//         if(p & 1) Ans = mul(Ans, base);
//         base = mul(base, base);
//         p >>= 1;
//     }
// }
```

```

//      }
// }

// int main() {
//      scanf("%d", &T);
//      while (T--) {
//          scanf("%d", &n);
//          if(n <= 3) {
//              printf("1\n");
//              continue;
//          }
//          init();
//          Qmat_pow(n);
//          printf("%lld\n", Ans.m[2][1]);
//      }
// }
/*
f[i-1]      f[i]
f[i-2] ----> f[i-1]
f[i-3]      f[i-2]
f[i] = f[i-1] * 1 + f[i-2] * 0 + f[i-3] * 1
f[i-1] = f[i-1] * 1 + f[i-2] * 0 + f[i-3] * 0
f[i-2] = f[i-1] * 0 + f[i-2] * 1 + f[i-3] * 0
so
1 0 1
1 0 0
0 1 0
*/
//P6569
#include<iostream>
#include<cstdio>
#include<cstring>
#include<algorithm>
using namespace std;
struct matrix{//用结构体存矩阵敲好使哒~
    long long qwq[105][105];
    int x,y;//x,y就是矩阵的行数和列数
    matrix(){x=0,y=0;memset(qwq,0,sizeof(qwq));};//构造函数，刚听说这玩意，用着玩玩
};
matrix operator * (const matrix &a,const matrix &b){//重载“异或版矩阵乘法”的运算符
    matrix c;
    c.x=a.x,c.y=b.y;
    for(int i=1;i<=a.x;i++){

```

```

        for(int j=1;j<=b.y;j++){
            for(int k=1;k<=a.y;k++){
                c.qwq[i][j]^=a.qwq[i][k]*b.qwq[k][j];//和矩阵乘法别无二致，只不过就是
            }
        }
        return c;
    }
}

matrix wyx[40];//存储e矩阵的2^i次方，取这个变量名是因为想借助神仙的力量AC本题
long long f[105];//每个城市的初始魔法值（提醒：十年OI一场空.....）
int main(){
    int n,m,q;
    scanf("%d%d%d",&n,&m,&q);
    for(int i=1;i<=n;i++)
        scanf("%lld",&f[i]);
    wyx[0].x=n,wyx[0].y=n;//初始化一开始的邻接矩阵（也就是2的0次方）的大小
    for(int i=1;i<=m;i++){
        int a,b;
        scanf("%d%d",&a,&b);
        wyx[0].qwq[a][b]=wyx[0].qwq[b][a]=1;//给邻接矩阵连上边
    }
    for(int i=1;i<32;i++)wyx[i]=wyx[i-1]*wyx[i-1];//利用快速幂的思想，处理出邻接矩阵2的i次方
    while(q--){
        long long x;
        scanf("%lld",&x);
        matrix ans;
        for(int i=1;i<=n;i++)ans.qwq[1][i]=f[i];//一开始就是初始f值
        ans.x=1,ans.y=n;
        for(int i=0;i<32;i++){//开始进行二进制拆分
            if((x>>i)&1)//如果这一位是1
                ans=ans*wyx[i];//就乘上对应的2的i次方
        }
        printf("%lld\n",ans.qwq[1][1]);
    }
    return 0;
}

```

并查集

```
//推导部分和
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll N=1e5+10;
struct tree{
    ll n,w,to;
}e[N<<1];
ll head[N<<1],tot,fa[N],vis[N],sum[N];
void add(int u,int v,ll w){
    e[tot].to=v;
    e[tot].n=head[u];
    e[tot].w=w;
    head[u]=tot++;
}
int find(int i){
    if(fa[i]!=i) return fa[i]=find(fa[i]);
    return i;
}
void unite(int x,int y){
    x=find(x);
    y=find(y);
    if(x!=y) fa[x]=y;
}
void bfs(int x){
    vis[x]=1;
    queue<int>q; q.push(x);
    sum[x]=0;
    while(!q.empty()){
        int u=q.front();
        q.pop();
        for(int i=head[u];i!=-1;i=e[i].n){
            int v=e[i].to;
            if(vis[v]) continue;
            vis[v]=1;
            sum[v]=sum[u]+e[i].w;
            q.push(v);
        }
    }
}
int main()
```

```

{
    memset(head, -1, sizeof(head));
    int n, m, q;
    cin >> n >> m >> q;
    for (int i = 1; i <= n; i++) fa[i] = i;
    for (int i = 1; i <= m; i++) {
        ll l, r, w;
        cin >> l >> r >> w;
        add(l - 1, r, w);
        add(r, l - 1, -w);
        unite(l - 1, r);
    }
    for (int i = 0; i <= n; i++) {
        if (!vis[i]) bfs(i);
    }
    for (int i = 1; i <= m; i++) {
        int l, r;
        cin >> l >> r;
        if (find(l - 1) != find(r)) cout << "UNKNOWN" << endl;
        else cout << sum[r] - sum[l - 1] << endl;
    }
    return 0;
}

```

FFT（快速傅里叶变换）

```
// 当vector用就可以了
#include <bits/stdc++.h>
#define fp(i, a, b) for (int i = (a), i##_ = (b) + 1; i < i##_; ++i)
#define fd(i, a, b) for (int i = (a), i##_ = (b) - 1; i > i##_; --i)
using namespace std;
using ll = int64_t;
using db = double;
/*-----*/
struct cp {
    db x, y;
    cp(db real = 0, db imag = 0) : x(real), y(imag) {};
    cp operator+(cp b) const { return {x + b.x, y + b.y}; }
    cp operator-(cp b) const { return {x - b.x, y - b.y}; }
    cp operator*(cp b) const { return {x * b.x - y * b.y, x * b.y + y * b.x}; }
};
using vcp = vector<cp>;
using Poly = vector<int>;
namespace FFT {
    const db pi = acos(-1);
    vcp Omega(int L) {
        vcp w(L);
        w[1] = 1;
        for (int i = 2; i < L; i <= 1) {
            auto w0 = w.begin() + i / 2, w1 = w.begin() + i;
            cp wn(cos(pi / i), sin(pi / i));
            for (int j = 0; j < i; j += 2)
                w1[j] = w0[j >> 1], w1[j + 1] = w1[j] * wn;
        }
        return w;
    }
    auto W = Omega(1 << 21); // NOLINT
    void DIF(cp *a, int n) {
        cp x, y;
        for (int k = n >> 1; k >= 1)
            for (int i = 0; i < n; i += k << 1)
                for (int j = 0; j < k; ++j)
                    x = a[i + j], y = a[i + j + k],
                    a[i + j + k] = (a[i + j] - y) * W[k + j], a[i + j] = x + y;
    }
    void IDIT(cp *a, int n) {
        cp x, y;
```

```

    for (int k = 1; k < n; k <= 1)
        for (int i = 0; i < n; i += k << 1)
            for (int j = 0; j < k; ++j)
                x = a[i + j], y = a[i + j + k] * W[k + j], a[i + j + k] = x - y,
                a[i + j] = x + y;
    const db Inv = 1. / n;
    fp(i, 0, n - 1) a[i].x *= Inv, a[i].y *= Inv;
    reverse(a + 1, a + n);
}
} // namespace FFT

namespace Polynomial {
// basic operator
void DFT(vcp &a) { FFT::DIF(a.data(), a.size()); }
void IDFT(vcp &a) { FFT::IDIT(a.data(), a.size()); }
int norm(int n) { return 1 << (__lg(n - 1) + 1); }

// Poly mul
vcp &dot(vcp &a, vcp &b) {
    fp(i, 0, a.size() - 1) a[i] = a[i] * b[i];
    return a;
}

Poly operator+(Poly a, Poly b) {
    int maxlen = max(a.size(), b.size());
    Poly ans(maxlen + 1);
    a.resize(maxlen + 1), b.resize(maxlen + 1);
    for (int i = 0; i < maxlen; i++) ans[i] = a[i] + b[i];
    return ans;
}

Poly operator*(ll k, Poly a) {
    Poly ans;
    for (auto i : a) ans.push_back(k * i);
    return ans;
}

Poly operator*(Poly a, Poly b) {
    int n = a.size() + b.size() - 1;
    vcp c(norm(n));
    fp(i, 0, a.size() - 1) c[i].x = a[i];
    fp(i, 0, b.size() - 1) c[i].y = b[i];
    DFT(c), dot(c, c), IDIT(c), a.resize(n);
    fp(i, 0, n - 1) a[i] = int(c[i].y * .5 + .5);
    return a;
}
}

```

```
} // namespace Polynomial  
/*-----*/  
using namespace Polynomial;
```


NTT (数论变换)

```
#include <bits/stdc++.h>
#define fp(i, a, b) for (int i = (a), i##_ = (b) + 1; i < i##_; ++i)
#define fd(i, a, b) for (int i = (a), i##_ = (b) - 1; i > i##_; --i)
using namespace std;
const int maxn = 2e5 + 5, P = 998244353;
using ll = int64_t;
#define ADD(a, b) (((a) += (b)) >= P ? (a) -= P : 0)
#define SUB(a, b) (((a) -= (b)) < 0 ? (a) += P : 0)
#define MUL(a, b) (ll(a) * (b) % P)
int POW(ll a, int b = P - 2, ll x = 1) {
    for (; b >= 1, a = a * a % P)
        if (b & 1) x = x * a % P;
    return x;
}

namespace NTT {
const int g = 3;
vector<int> Omega(int L) {
    int wn = POW(g, P / L);
    vector<int> w(L);
    w[L >> 1] = 1;
    fp(i, L / 2 + 1, L - 1) w[i] = MUL(w[i - 1], wn);
    fd(i, L / 2 - 1, 1) w[i] = w[i << 1];
    return w;
}
auto W = Omega(1 << 21);
void DIF(int *a, int n) {
    for (int k = n >> 1; k; k >= 1) {
        for (int i = 0, y; i < n; i += k << 1)
            fp(j, 0, k - 1) y = a[i + j + k],
                a[i + j + k] = MUL(a[i + j] - y + P, W[k + j]),
                ADD(a[i + j], y);
    }
}
void IDIT(int *a, int n) {
    for (int k = 1; k < n; k <= 1) {
        for (int i = 0, x, y; i < n; i += k << 1)
            fp(j, 0, k - 1) x = a[i + j], y = MUL(a[i + j + k], W[k + j]),
                a[i + j + k] = x - y < 0 ? x - y + P : x - y,
                ADD(a[i + j], y);
    }
}
```

```

    int Inv = P - (P - 1) / n;
    fp(i, 0, n - 1) a[i] = MUL(a[i], Inv);
    reverse(a + 1, a + n);
}

} // namespace NTT
namespace Polynomial {
using Poly = std::vector<int>;
Poly &operator*=(Poly &a, int b) {
    for (auto &x : a) x = MUL(x, b);
    return a;
}
Poly operator*(Poly a, int b) { return a *= b; }
Poly operator*(int a, Poly b) { return b * a; }
void DFT(Poly &a) { NTT::DIF(a.data(), a.size()); }
void IDFT(Poly &a) { NTT::IDIT(a.data(), a.size()); }
int norm(int n) { return 1 << (32 - __builtin_clz(n - 1)); }
void norm(Poly &a) {
    if (!a.empty()) a.resize(norm(a.size()), 0);
}
Poly &dot(Poly &a, Poly &b) {
    fp(i, 0, a.size() - 1) a[i] = MUL(a[i], b[i]);
    return a;
}
Poly operator*(Poly a, Poly b) {
    int n = a.size() + b.size() - 1, L = norm(n);
    if (a.size() <= 8 || b.size() <= 8) {
        Poly c(n);
        fp(i, 0, a.size() - 1) fp(j, 0, b.size() - 1) c[i + j] =
            (c[i + j] + (ll)a[i] * b[j]) % P;
        return c;
    }
    a.resize(L), b.resize(L);
    DFT(a), DFT(b), dot(a, b), IDFT(a);
    return a.resize(n), a;
}
} // namespace Polynomial
using namespace Polynomial;

```

扫描线算法

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll arr[15010],n,height[15010];

struct node{
    ll l,r,h;
}a[15010];

struct compare{
    bool operator()(pair<ll,ll> a,pair<ll,ll> b){
        if(a.first!=b.first) return a.first<b.first;
        return a.second>b.second;
        //大根堆
    }
};

bool cmp(node x,node y){
    return x.l<y.l;
}

int main(){
    ll l,r,h,m=0;
    while(scanf("%lld %lld %lld",&l,&h,&r)!=EOF){
        a[++m].l=l;
        a[m].r=r-1;
        a[m].h=h;
        arr[++n]=l;
        arr[++n]=r;
        arr[++n]=r-1;
    }
    sort(arr+1,arr+n+1);
    ll len=unique(arr+1,arr+n+1)-arr-1;
    for(int j=1;j<=m;j++){
        a[j].l=lower_bound(arr+1,arr+len+1,a[j].l)-arr;
        a[j].r=lower_bound(arr+1,arr+len+1,a[j].r)-arr;
    }
    sort(a+1,a+m+1,cmp);
    priority_queue<pair<ll,ll>, vector<pair<ll,ll>>, compare> heap;
    for(int i=1,j=0;i<=len;i++){
        for(;j<=m&& a[j].l<=i;j++){
            heap.push(make_pair(a[j].h,a[j].r));
        }
    }
```

```

while(!heap.empty() && heap.top().second < i)
{
    heap.pop();
}

if(!heap.empty()){
    height[i] = heap.top().first;
}
}

ll ansn = 0;
pair<ll, ll> ans[15010];
for(ll i = 1, pre = 0; i <= len; i++){
    if(pre != height[i]){
        ans[++ansn] = make_pair(arr[i], height[i]);
    }
    pre = height[i];
}
for(int i = 1; i <= ansn; i++){
    cout << ans[i].first << ' ' << ans[i].second << ' ';
}
}

```

最长上升子序列

```
#include<cstdio>
#include<algorithm>
const int MAXN=200001;
int a[MAXN];
int d[MAXN];
int main()
{
    int n;
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
        scanf("%d",&a[i]);
    d[1]=a[1];
    int len=1;
    for(int i=2;i<=n;i++)
    {
        if(a[i]>d[len])
            d[++len]=a[i];
        else
        {
            int j=std::lower_bound(d+1,d+len+1,a[i])-d;
            d[j]=a[i];
        }
    }
    printf("%d\n",len);
    return 0;
}
```

全排列输出

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> arr = {1, 2, 3}; // 示例数组

    // 先排序（必须步骤，确保生成所有排列）
    std::sort(arr.begin(), arr.end());

    // 输出所有排列组合
    do {
        for (int num : arr) {
            std::cout << num << " ";
        }
        std::cout << "\n";
    } while (std::next_permutation(arr.begin(), arr.end()));

    return 0;
}
```

位运算用法

/*为了更好的理解状压dp，首先介绍位运算相关的知识。

'&'符号， $x \& y$ ，会将两个十进制数在二进制下进行与运算（都1为1，其余为0）然后返回其十进制下的值。例如3(11

'|'符号， $x | y$ ，会将两个十进制数在二进制下进行或运算（都0为0，其余为1）然后返回其十进制下的值。例如3(1

'^'符号， $x \wedge y$ ，会将两个十进制数在二进制下进行异或运算（不同为1，其余为0）然后返回其十进制下的值。例如：

'~'符号， $\sim x$ ，按位取反。例如 $\sim 101 = 010$ 。

'<<'符号，左移操作， $x \ll 2$ ，将x在二进制下的每一位向左移动两位，最右边用0填充， $x \ll 2$ 相当于让x乘以4。 '>>'

1.判断一个数字x二进制下第i位是不是等于1。（最低第1位）

方法：if(((1<<(i-1))&x)>0) 将1左移i-1位，相当于制造了一个只有第i位 上是1，其他位上都是0的二进制数。

2.将一个数字x二进制下第i位更改成1。

方法： $x = x | (1 \ll (i-1))$ 证明方法与1类似。

3.将一个数字x二进制下第i位更改成0。

方法： $x = x \& \sim (1 \ll (i-1))$

4.把一个数字二进制下最靠右的第一个1去掉。

方法： $x = x \& (x-1)$

__builtin_popcount() 二进制数中一的个数

*/

```
#include<iostream>
```

```
using namespace std;
```

```
struct state {
```

```
    int st[1 << 12+1];
```

```
    int num;
```

```
} a[15];
```

```
int m, n;
```

```
int f[15][1 << 12+1];
```

```
const int MOD = 100000000;
```

```
void getstate(int i, int t) {
```

```
    a[i].num = 0;
```

```
    for (int j = 0; j < (1 << n); j++) {
```

```
        if ((j & (j << 1)) || (j & t))
```

```
            continue;
```

```
        a[i].st[++a[i].num] = j;
```

```
    }
```

```
}
```

```

void dp() {
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= a[i].num; j++)
            f[i][j] = 0;
    for (int j = 1; j <= a[1].num; j++)
        f[1][j] = 1;
    for (int i = 2; i <= m; i++) {
        for (int j = 1; j <= a[i].num; j++) {
            int s1 = a[i].st[j];
            for (int k = 1; k <= a[i-1].num; k++) {
                int s2 = a[i-1].st[k];
                if (s1 & s2) continue;
                f[i][j] = (f[i][j] + f[i-1][k]) % MOD;
            }
        }
    }
    int ans = 0;
    for (int j = 1; j <= a[m].num; j++)
        ans = (ans + f[m][j]) % MOD;
    cout << ans << endl;
}

int main() {
    cin >> m >> n;
    for (int i = 1; i <= m; i++) {
        int t = 0;
        for (int j = 0; j < n; j++) {
            int x;
            cin >> x;
            t = (t << 1) + (1 - x);
        }
        getstate(i, t);
    }
    dp();
    return 0;
}

```


Map的使用

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    //什么是map? map是干什么用的
    map<int,int>mp; //定义了一个名叫mp的map容器
    mp[10001]=82;mp[100002]=82;
    //输出10001信息
    cout<<mp[10001]<<endl;//可能会溢出
    //安全的方法
    if(mp.find(10005)!=mp.end()){
        cout<<mp[10005]<<endl;
    }

    else cout<<"没有此人信息"<<endl;
    //储存学生姓名对应成绩
    map<string,int>stu;
    stu["xiaoming"]=85;
    stu["xiaohong"]=77;
    cout<<stu["xiaohong"]<<endl;
    //map的遍历
    //1.
    for(auto it:stu){//将容器从头到尾遍历了一遍
        //map自动按照首位字典序从小到大排序
        cout<<it.first<<' '<<it.second<<endl;
    }
    //2.迭代器类型
    map<string,int>::iterator it;//定义map<string,int>类型的迭代器it
    for(it=stu.begin();it!=stu.end();it++){
        cout<<(*it).first<<' '<<(*it).second<<endl;
    }
}
```

优先队列（堆）

//自定义类的优先队列

```
struct Student {  
    std::string name;  
    int score;  
    int age;
```

// 重载 < 运算符，用于大根堆（按score从大到小）

```
bool operator<(const Student& other) const {  
    // 注意：priority_queue默认用 < 比较，但实际表现是最大堆  
    // 所以这里返回 true 表示优先级更低  
    return score < other.score; // 大根堆  
}
```

```
};
```

```
priority_queue<Student>q1;//大根堆
```

```
priority_queue<int>q2;//默认大根堆
```

```
priority_queue<int,vector<int>,greater<int>>q3;//小根堆
```

扫描线+线段树（矩形面积并）

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll arr[15010],n,height[15010];

struct node{
    ll l,r,h;
}a[15010];

struct compare{
    bool operator()(pair<ll,ll> a,pair<ll,ll> b){
        if(a.first!=b.first) return a.first<b.first;
        return a.second>b.second;
        //大根堆
    }
};

bool cmp(node x,node y){
    return x.l<y.l;
}

int main(){
    ll l,r,h,m=0;
    while(scanf("%lld %lld %lld",&l,&h,&r)!=EOF){
        a[++m].l=l;
        a[m].r=r-1;
        a[m].h=h;
        arr[++n]=l;
        arr[++n]=r;
        arr[++n]=r-1;
    }
    sort(arr+1,arr+n+1);
    ll len=unique(arr+1,arr+n+1)-arr-1;
    for(int j=1;j<=m;j++){
        a[j].l=lower_bound(arr+1,arr+len+1,a[j].l)-arr;
        a[j].r=lower_bound(arr+1,arr+len+1,a[j].r)-arr;
    }
    sort(a+1,a+m+1,cmp);
    priority_queue<pair<ll,ll>, vector<pair<ll,ll>>, compare> heap;
    for(int i=1,j=0;i<=len;i++){
        for(;j<=m&& a[j].l<=i;j++){
            heap.push(make_pair(a[j].h,a[j].r));
        }
    }
}
```

```

while(!heap.empty() && heap.top().second < i)
{
    heap.pop();
}

if(!heap.empty()){
    height[i] = heap.top().first;
}
}

ll ansn = 0;
pair<ll, ll> ans[15010];
for(ll i = 1, pre = 0; i <= len; i++){
    if(pre != height[i]){
        ans[++ansn] = make_pair(arr[i], height[i]);
    }
    pre = height[i];
}
for(int i = 1; i <= ansn; i++){
    cout << ans[i].first << ' ' << ans[i].second << ' ';
}
}

```

建树参考

```
#include<bits/stdc++.h>
// #define int long long
#define in(a) a = read_int()
using namespace std;
const int N = 2e5 + 5;
const int inf = INT_MAX;
inline int read_int() {
    int x = 0;
    char ch = getchar();
    bool f = 0;
    while('9' < ch || ch < '0') f |= ch == '-' , ch = getchar();
    while('0' <= ch && ch <= '9') x = (x << 3) + (x << 1) + ch - '0' , ch = getchar();
    return f ? -x : x;
}

struct Edge {
    int n , v , w;
} edge[N << 1];
int head[N] , eid;
void eadd(int u , int v , int w) {
    edge[++ eid].n = head[u] , edge[eid].v = v , edge[head[u] = eid].w = w;
}

int n;
int l[N] , r[N] , ans[N];
void dfs(int u , int fa) { // 树形 DP 部分
    l[u] = 1; // 初始化左边界
    for(int i = head[u] ; i ; i = edge[i].n) {
        int v = edge[i].v , w = edge[i].w;
        if(v != fa) {
            r[v] = w - 1; // 初始化右边界
            dfs(v , u);
            r[u] = min(r[u] , w - l[v]); // 转移过程
            l[u] = max(l[u] , w - r[v]);
        }
    }
}

queue<int>q;
signed main() {
    in(n);
    for(int i = 1 ; i < n ; i ++) {
        int u , v , w;
        in(u) , in(v) , in(w);
```

```

        eadd(u , v , w) , eadd(v , u , w);
    }
    r[1] = 1e9; // 初始化根节点的右边界
    dfs(1 , 0); // 求每个点权的取值范围
    for(int i = 1 ; i <= eid ; i += 2) { // 判无解部分
        int u = edge[i].v , v = edge[i + 1].v , w = edge[i].w;
        if(r[u] + r[v] < w || l[u] > r[u] || l[v] > r[v]) {
            cout<<"NO";

                return 0;

        }
    }
    ans[1] = l[1]; // 若有解，根节点点权取左边界
    q.push(1);
    while(!q.empty()) { // 广搜部分
        int u = q.front();
        q.pop();
        for(int i = head[u] ; i ; i = edge[i].n) {
            int v = edge[i].v , w = edge[i].w;
            if(ans[v]) continue;
            ans[v] = w - ans[u]; // 递推求解
            q.push(v);
        }
    }
    cout<<"YES\n"; // 打印答案
    for(int i = 1 ; i <= n ; i ++) cout<<ans[i]<<' ';
    return 0;
}

```

Hash树

//树同构

```
#include <bits/stdc++.h>

typedef long long LL;
const int N = 60, MO = 998244353;

struct Edge {
    int nex, v;
}edge[N << 1]; int tp;

int e[N], n, m, turn, fr[N], p[1000010], top, siz[N], h[N], g[N];
std::vector<int> v[N];
bool vis[1000010];

inline void add(int x, int y) {
    tp++;
    edge[tp].v = y;
    edge[tp].nex = e[x];
    e[x] = tp;
    return;
}

inline bool equal(int a, int b) {
    int len = v[a].size();
    if(len != v[b].size()) return false;
    for(int i = 0; i < len; i++) {
        if(v[a][i] != v[b][i]) return false;
    }
    return true;
}

inline void getp(int n) {
    for(int i = 2; i <= n; i++) {
        if(!vis[i]) {
            p[++top] = i;
        }
        for(int j = 1; j <= top && i * p[j] <= n; j++) {
            vis[i * p[j]] = 1;
            if(i % p[j] == 0) {
                break;
            }
        }
    }
}
```

```

    }
}
return;
}

void DFS_1(int x, int f) {
    siz[x] = 1;
    h[x] = 1;
    for(int i = e[x]; i; i = edge[i].nex) {
        int y = edge[i].v;
        if(y == f) continue;
        DFS_1(y, x);
        h[x] = (h[x] + 1ll * h[y] * p[siz[y]] % MO) % MO;
        siz[x] += siz[y];
    }
    return;
}

void DFS_2(int x, int f, int V) {
    g[x] = (h[x] + 1ll * V * p[n - siz[x]] % MO) % MO;
    v[turn].push_back(g[x]);
    V = (1ll * V * p[n - siz[x]] % MO + 1) % MO;
    for(int i = e[x]; i; i = edge[i].nex) {
        int y = edge[i].v;
        if(y == f) {
            continue;
        }
        DFS_2(y, x, ((LL)V + h[x] - 1 - 1ll * h[y] * p[siz[y]] % MO + MO) % MO);
    }
    return;
}

int main() {
    getp(1000009);
    scanf("%d", &m);
    for(turn = 1; turn <= m; turn++) {
        scanf("%d", &n);
        tp = 0;
        memset(e + 1, 0, n * sizeof(int));
        for(int i = 1, x; i <= n; i++) {
            scanf("%d", &x);
            if(x) {

```



```

        add(x, i);
        add(i, x);
    }
}
DFS_1(1, 0);
DFS_2(1, 0, 0);
std::sort(v[turn].begin(), v[turn].end());
}

for(int i = 1; i <= m; i++) {
    fr[i] = i;
}
for(int i = 2; i <= m; i++) {
    for(int j = 1; j < i; j++) {
        if(equal(i, j)) {
            fr[i] = fr[j];
            break;
        }
    }
}
}
for(int i = 1; i <= m; i++) {
    printf("%d\n", fr[i]);
}
return 0;
}

```